# Mcu IAP

## IAP简介

一般情况下对芯片进行固件烧写都是使用芯片定义好的编程口SWD或者JTAG，如果想要使用串口烧写固件需要修改BOOT0和BOOT1的引脚接线，关于BOOT引脚不同接线下芯片复位后如何启动可参考这篇文章。{关于stm32程序烧写BOOT1和BOOT0的设置问题_什么样的芯片需要烧写boot1-CSDN博客}

IAP的全程为In application programing，即在应用程序编程。主要作用就是在产品开发完成以后方便对产品的固件进行升级，一般情况下产品都是使用外壳封装好的，极少数会将程序烧录调试口SWD和JTAG开放出来，这时候就需要通过预留的通信接口对固件进行升级，如果产品有无线通信协议可以使用更为高级的OTA对固件升级。

IAP一般需要两段程序实现，第一段叫Bootloader，即引导加载程序，第二段叫APP，即我们的实现业务功能应用程序。所以要在产品实现IAP功能需要我们在出厂前先烧录Bootloader程序到产品中，一般情况下是烧录在芯片的flash上。

Bootloader程序主要就是做三件事：

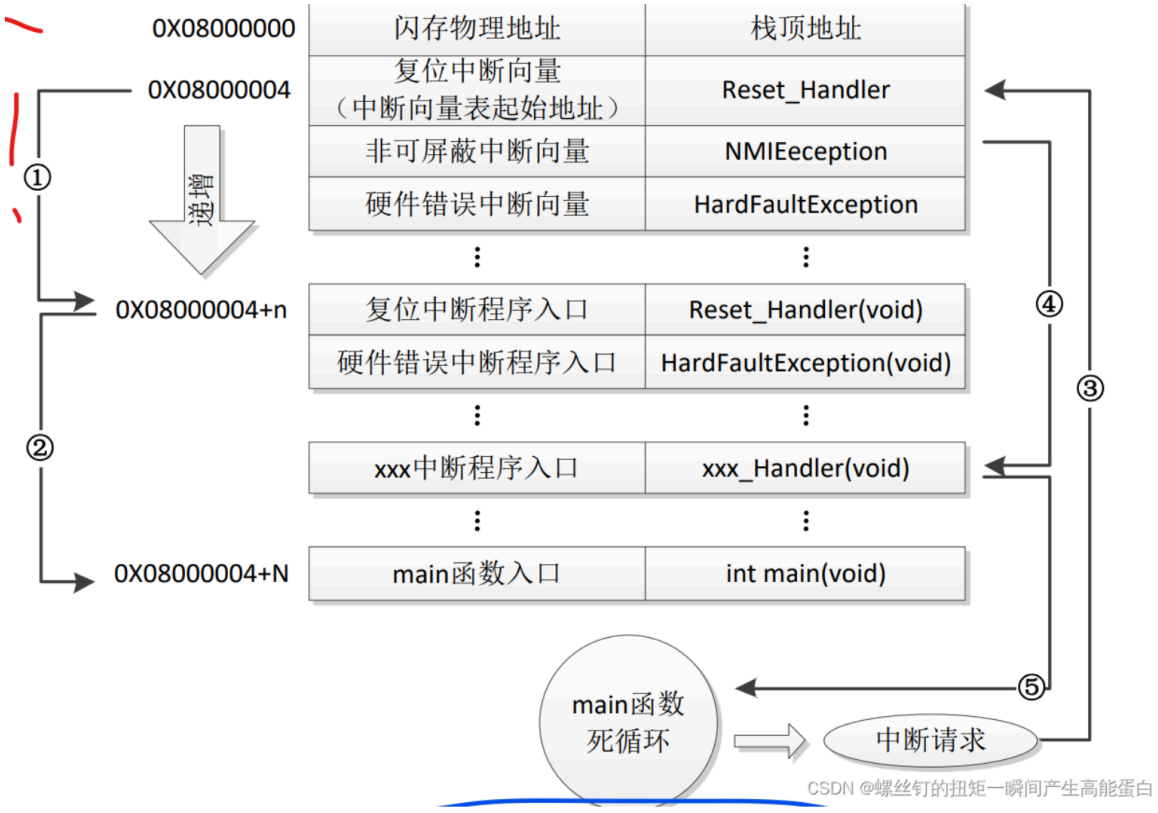
- 接收要更新的程序暂存到RAM
- 将RAM的数据写入到flash（当然也可以将APP拷贝在SRAM上运行，但是这样每次开机都要重新刷固件，而且一般MCU的SRAM都比较小，对于要实现业务功能的APP不推荐在SRAM上运行）
- 程序跳转到APP

## 芯片闪存和程序启动

### 闪存和启动

要实现IAP必须要对芯片的闪存设计有一定的了解。我们以STM32F10X为例看一下芯片的闪存模块和正常程序启动流程。

| 块 | 名称 | 地址范围 | 长度(字节) |
|---|---|---|---|
| 主存储器 | 页0 | 0x0800 0000 – 0x0800 03FF | 1K |
| | 页1 | 0x0800 0400 – 0x0800 07FF | 1K |
| | 页2 | 0x0800 0800 – 0x0800 0BFF | 1K |
| | 页3 | 0x0800 0C00 – 0x0800 0FFF | 1K |
| | 页4 | 0x0800 1000 – 0x0800 13FF | 1K |
| | .<br>.<br>. | .<br>.<br>. | .<br>.<br>. |
| | 页127 | 0x0801 FC00 – 0x0801 FFFF | 1K |
| 信息块 | 启动程序代码 | 0x1FFF F000 – 0x1FFF F7FF | 2K |
| | 用户选择字节 | 0x1FFF F800 – 0x1FFF F80F | 16 |
| 闪存存储器接口寄存器 | FLASH_ACR | 0x4002 2000 – 0x4002 2003 | 4 |
| | FLASH_KEYR | 0x4002 2004 – 0x4002 2007 | 4 |
| | FLASH_OPTKEYR | 0x4002 2008 – 0x4002 200B | 4 |
| | FLASH_SR | 0x4002 200C – 0x4002 200F | 4 |
| | FLASH_CR | 0x4002 2010 – 0x4002 2013 | 4 |
| | FLASH_AR | 0x4002 2014 – 0x4002 2017 | 4 |
| | 保留 | 0x4002 2018 – 0x4002 201B | 4 |
| | FLASH_OBR | 0x4002 201C – 0x4002 201F | 4 |
| | FLASH_WRPR | 0x4002 2020 – 0x4002 2023 | 4 |




当烧录一个程序，程序文件一般是写入flash起始地址0x08000000中的。当单片机上电以后，首先会将flash的前四个字节写入栈顶地址，也就是堆栈指针。这样做的目的是初始化堆栈指针，使其能够安全调用函数和中断服务。接下来会将0x08000004这里写入复位中断向量的地址，当单片机上电复位后，会触发复位中断，此时PC会根据中断向量表中的地址去执行中断服务函数，如上图步骤一所示；接下来在复位中断服务函数中会进行必要的时钟初始化，然后调用用户程序的main函数，此时会进入用户程序的main函数处继续执行程序。

我们打开项目工程的启动文件在向量表可以看到和上图是对应的和复位中断可以看到和上面的流程是对应的

```
__Vectors       DCD     __initial_sp                    ; Top of Stack
                DCD     Reset_Handler                   ; Reset Handler
                DCD     NMI_Handler                     ; NMI Handler
                DCD     HardFault_Handler               ; Hard Fault Handler
                DCD     0                               ; Reserved
                DCD     0                               ; Reserved
                DCD     0                               ; Reserved
                DCD     0                               ; Reserved
                DCD     0                               ; Reserved
                DCD     0                               ; Reserved
                DCD     0                               ; Reserved
                DCD     SVC_Handler                     ; SVCall Handler
                DCD     0                               ; Reserved
                DCD     0                               ; Reserved
                DCD     PendSV_Handler                  ; PendSV Handler
                DCD     SysTick_Handler                 ; SysTick Handler

                ; External Interrupts
                DCD     WWDG_IRQHandler                 ; Window Watchdog
                DCD     PVD_IRQHandler                  ; PVD through EXTI Line detect
                DCD     RTC_IRQHandler                  ; RTC through EXTI Line
```

```
; Reset handler routine
Reset_Handler    PROC
                 EXPORT  Reset_Handler                [WEAK]
        IMPORT   __main
        IMPORT   SystemInit
                 LDR     R0, =SystemInit
                 BLX     R0
                 LDR     R0, =__main
                 BX      R0
                 ENDP
```

## 向量表偏移

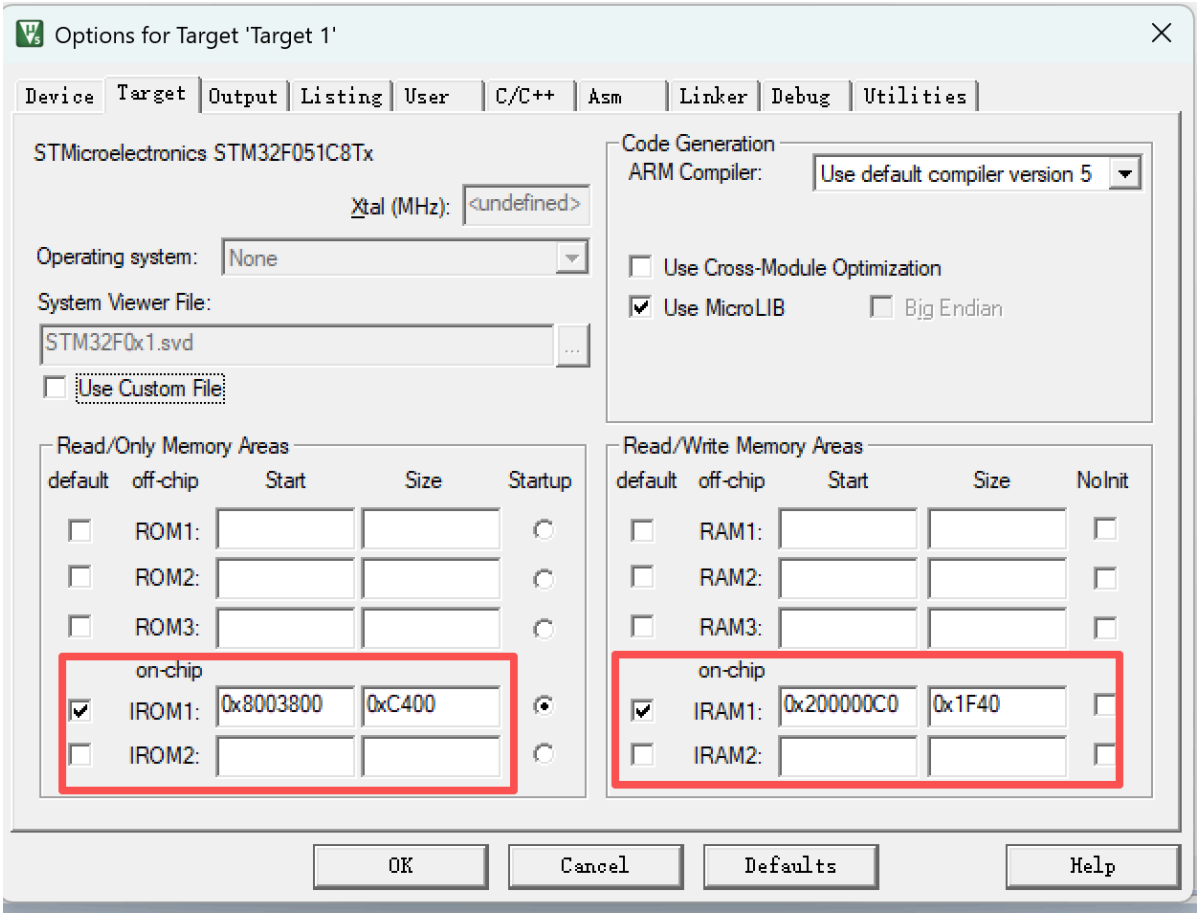
但是，有一个新问题摆在我们面前，那就是当我们程序跳转过后如果APP触发了中断就会去Bootloader的向量表执行中断服务函数，当执行完中断服务函数后，再回到APP会因为栈地址被破坏导致上下文不匹配，程序就直接跑飞了。

因此跳转到APP我们第一个要做的事情就是将中断向量表映射到另一个地址或者直接偏移到APP程序首地址。这里Cortex-M0由于厂商没有定义向量偏移寄存器VTOR，无法动态修改向量表的地址，只能使用向量表映射的方法。

```c
//下面是两种方法的实现
//向量表偏移
SCB->VTOR == 0x08008000;    //0x08008000就是APP的地址

//向量表映射
void JumpToApp_SetVectorTable(void)
{
    __disable_irq();
    memcpy((void *)RAM_START_ADDR, (void *)ApplicationAddress, VECTOR_SIZE);
    SYSCFG_MemoryRemapConfig(SYSCFG_MemoryRemap_SRAM);
    set_boot_flag(BOOT_APP);
    __enable_irq();
}
```

在KEIL工程中还需要做下面的选项配置

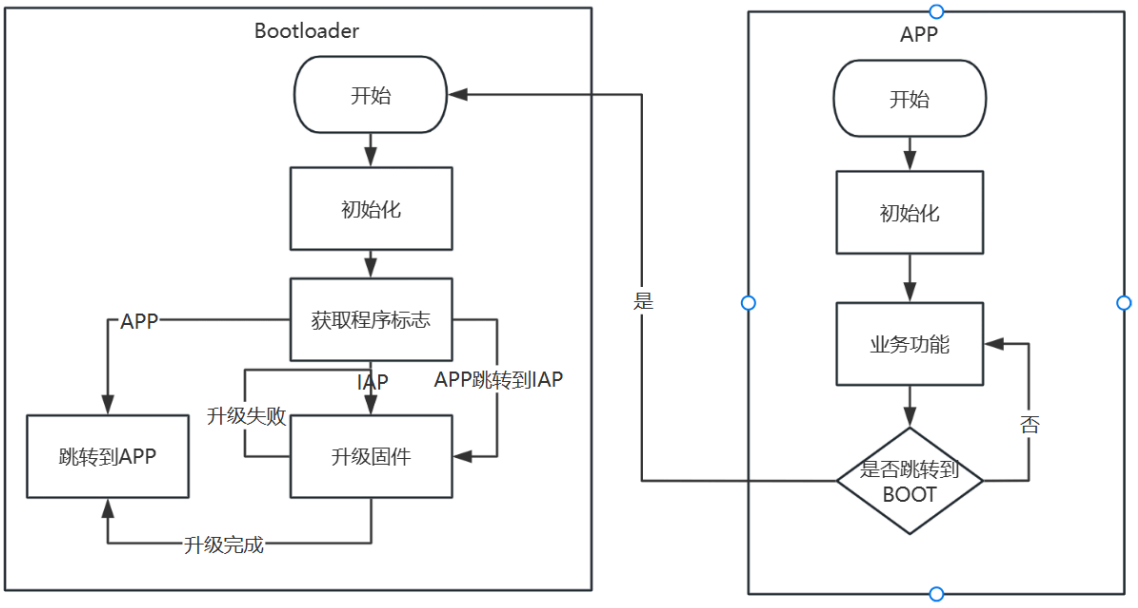



## Bootloader程序实现

```
//程序跳转
//设置栈指针、设置PC寄存器为Reset_Handle
typedef void (*pFunction)(void);

void jump_code(uint32_t is_app)
{
    DBG("Jump to %s!\n",is_app?"app":"bootloader");
    if(is_app) HAL_UART_DeInit(&huart5);
    uint32_t JumpAddress = *(__IO uint32_t*) ((is_app?
APPLICATION_ADDRESS:BOOTLOADER_ADDRESS) + 4);
    pFunction JumpToApplication = (pFunction) JumpAddress;
     __set_MSP(*(__IO uint32_t*) (is_app?
APPLICATION_ADDRESS:BOOTLOADER_ADDRESS));
    JumpToApplication();
}
```

# 指南IAP流程


Bootloader
开始
初始化
获取程序标志
APP
IAP
APP跳转到IAP
升级失败
跳转到APP
升级固件
升级完成
是
APP
开始
初始化
业务功能
否
是否跳转到
BOOT